

OPENRECEIVER

The single source-of-truth for building OpenReceiver with agentic coding workflows.

AirPlay = transport • OpenReceiver = experience • LRCLIB + local LRC • ₹0 infrastructure target

1. Executive Product Decision

OpenReceiver is a free, open-source AirPlay receiver targeting Android TV / Google TV and Windows. The product is deliberately more than a protocol receiver: its differentiator is a premium, cinematic music experience built around the metadata, artwork and playback state exposed by a legitimate AirPlay session.

PRODUCT THESIS — AirPlay is the transport. OpenReceiver is the experience.

The project is local-first and hobby-project friendly. There is no mandatory cloud backend, database, account, subscription, domain, paid API or hosted infrastructure. Core receiving should work over the local network.

The first release must prioritize reliable AirPlay audio reception and an exceptional music UI. Mirroring, photos and video are expansion work only after the music path is stable.

Automatic lyrics are an enhancement layer. AirPlay should not be treated as a guaranteed source of lyric text. OpenReceiver derives track identity and playback position from the AirPlay session and resolves lyrics independently.

2. Final Product Scope

2.1 In scope

- AirPlay receiver discovery and session handling on the local network.
- Supported AirPlay audio reception and playback.
- Track metadata: title, artist, album, artwork, duration and other valid exposed metadata.
- Playback state: playing, paused, stopped, position, duration, volume and rate where exposed.
- Premium cinematic Now Playing UI.
- Optional synchronized Lyrics mode.
- LRCLIB as the first automatic free/keyless lyrics provider.
- Local .LRC files as deterministic offline fallback.
- Windows desktop productization.
- Android TV / Google TV productization.
- Diagnostics, reconnect handling, caching and regression tests.
- Later: screen mirroring, photos and compatible video scenarios.

2.2 Explicitly out of scope

- DRM/FairPlay bypass, circumvention or decryption.
- Building an iOS sender application.
- Mandatory cloud relay, hosted backend or database.
- Paid lyrics API as a required dependency.
- Android phone/macOS as first-class v1 receiver platforms.
- Scraping protected lyrics websites or bypassing anti-bot protections as a required mechanism.
- Any claim of Apple endorsement, certification or affiliation.

3. Target User Experience

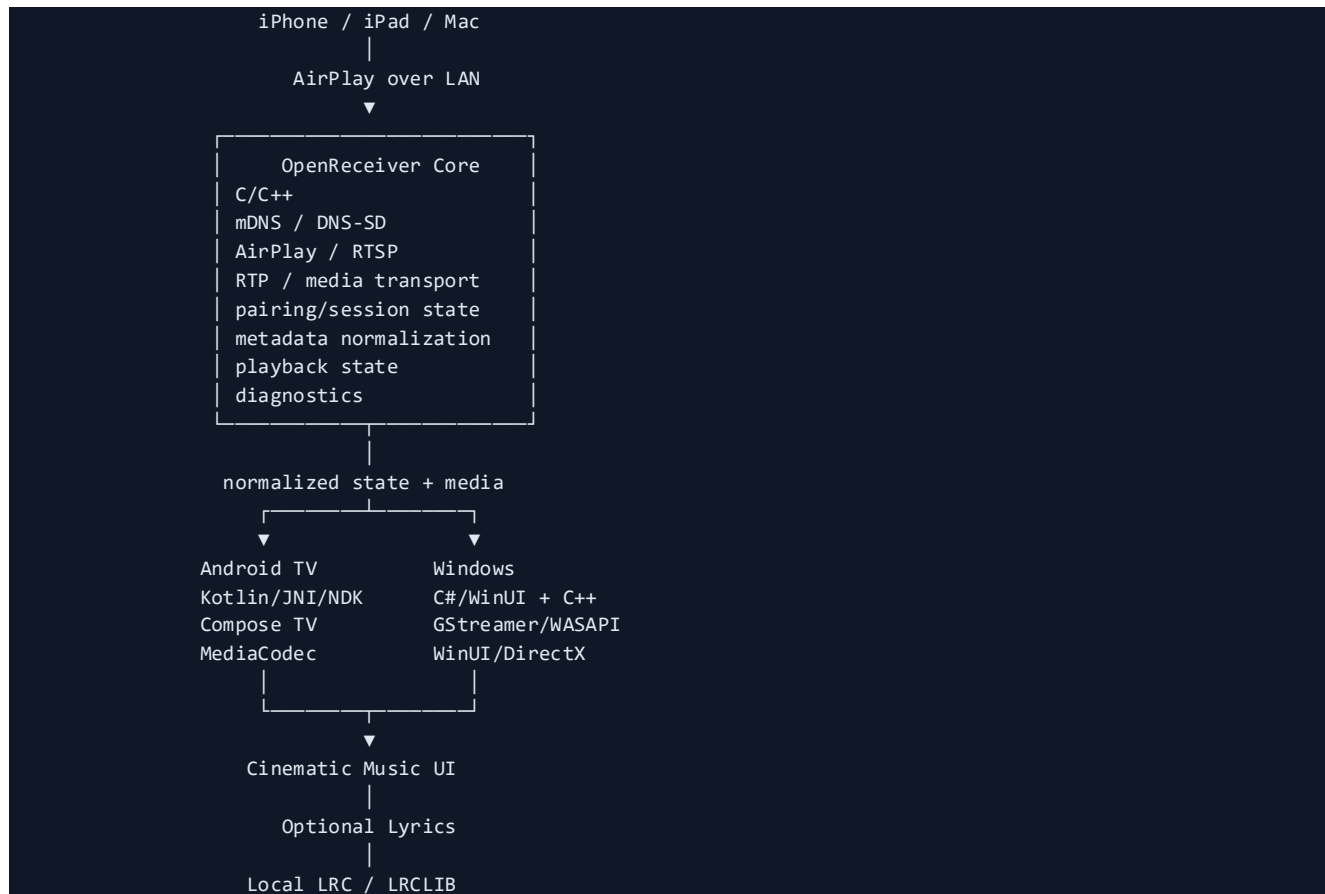
The user should not feel that they are operating a protocol debugger. The intended journey is:

1. Install OpenReceiver on a Windows PC or Android TV / Google TV.
2. OpenReceiver advertises itself as an AirPlay destination.
3. Select OpenReceiver from the sender's AirPlay picker.

4. Music starts playing on the receiver.
5. Album artwork becomes the visual anchor and drives the cinematic background.
6. Title, artist and album appear with strong hierarchy.
7. Playback controls remain minimal and can auto-hide.
8. Lyrics can be enabled without affecting audio if they are unavailable.

UI RULE — Large artwork + artwork-derived cinematic background + spacious composition + optional lyrics as a secondary visual area + minimal necessary controls.

4. Final System Architecture



4.1 Shared native core

- C/C++ implementation for protocol-sensitive and media-sensitive functionality.
- mDNS/DNS-SD advertisement and discovery integration.
- AirPlay session negotiation/lifecycle, RTSP/RTP handling and pairing/session state.
- Metadata parsing and normalization.
- Playback timeline/state tracking.
- Media transport and diagnostics.
- Stable platform/JNI bridge.
- No UI toolkit dependencies in the core.

4.2 Android TV / Google TV shell

- Kotlin application layer.
- JNI + Android NDK + CMake bridge to the native core.

- MediaCodec for appropriate media paths.
- AudioTrack or another suitable Android audio output.
- Jetpack Compose / Compose for TV.
- SurfaceView/TextureView for video paths.
- TV remote navigation, focus and distance-readable typography.

4.3 Windows shell

- C# + WinUI 3 application layer.
- Native C++ bridge/core.
- GStreamer where a flexible media pipeline is appropriate.
- WASAPI for Windows audio output.
- WinUI Composition / DirectX-capable rendering where useful.
- Tray/background lifecycle and installable packaging.

5. Canonical Application State

```

TrackInfo
├─ title
├─ artist
├─ album
├─ artworkUri / artworkBytes
├─ duration
├─ trackId / persistentId (when available)
├─ albumId / artistId (when available)
└─ quality / codec hints (only when valid)

PlaybackState
├─ status: playing | paused | stopped
├─ position
├─ duration
├─ volume
└─ rate (when exposed)

ReceiverState
├─ ready
├─ discovering
├─ connecting
├─ receiving
├─ reconnecting
├─ error
└─ disconnected

```

UI must consume these normalized models rather than raw protocol fields. Missing metadata is normal and must be handled gracefully.

6. AirPlay Metadata vs. Lyrics — Critical Decision

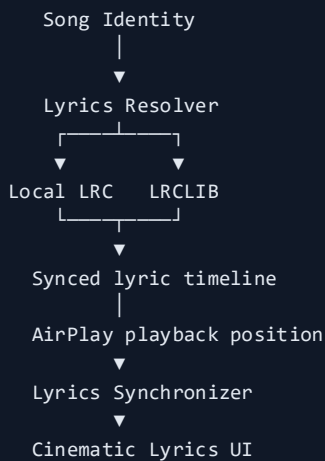
DO NOT DESIGN A LYRICS SYSTEM AROUND THE ASSUMPTION THAT APPLE MUSIC LYRIC TEXT IS SENT AS GUARANTEED AIRPLAY RECEIVER METADATA.

The receiver can obtain track identity and playback position from the AirPlay session. Apple Music's sender-side synchronized lyrics do not become a guaranteed receiver-side lyrics field. Therefore lyrics are a separate resolution layer.

```

AirPlay session
├─ title / artist / album / duration / artwork
└─ playback position

```



7. Final Lyrics Architecture

7.1 Provider abstraction

```
interface LyricsProvider {
    LyricsResult findLyrics(TrackInfo track);
}
```

Providers:

1. LocalLrcProvider
2. LrclibProvider
3. FutureProvider (optional)

LyricsResult:

- status
- matchedTrack
- source
- plainLyrics (optional)
- syncedLyrics (optional)
- confidence / match quality
- non-fatal error

7.2 LRCLIB — automatic baseline

- Preferred automatic provider for the free/open-source baseline.
- No paid API key should be required for the baseline implementation.
- Search using title + artist + album + duration where available.
- Prefer strong matches; reject risky remix/live/version mismatches.
- Use asynchronous requests, timeouts and local caching.
- Provider outage must never stop playback.

7.3 Local .LRC — guaranteed fallback

- Works offline.
- User can supply/import LRC files where the platform permits.
- Parser converts timestamps into an ordered lyric timeline.
- Parsed results can be cached.
- Used when online lyrics are disabled/unavailable/missing.

7.4 Synchronization rules

- AirPlay playback position is the synchronization clock.

- Pause freezes the active line.
- Seek immediately recalculates the active line.
- Track changes rebuild lyric state.
- Reconnects resynchronize from the current playback position.
- Plain unsynched lyrics are explicitly treated as lower-quality and must not pretend to be time-synchronized.

8. Premium Music UI Specification

8.1 Now Playing — default

- 16:9-first full-screen composition.
- Large album artwork on left/center-left.
- Artwork-derived blurred/darkened gradient background.
- Strong title → artist → album hierarchy.
- Thin progress/timeline indicator.
- Previous, play/pause, next, volume, more and Lyrics actions.
- Controls auto-hide after inactivity.
- Fullscreen/close/settings remain unobtrusive.
- Quality/codec indicator only when valid.

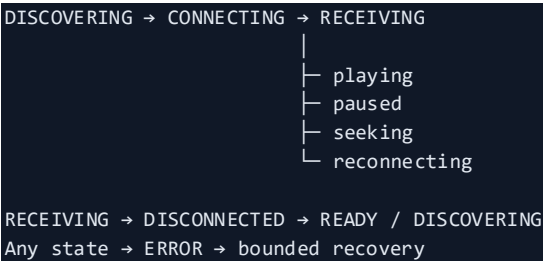
8.2 Lyrics mode

- Artwork remains a visual anchor.
- Lyrics occupy the secondary/right visual area in wide layouts.
- Current line is prominent; surrounding lines are quieter.
- Smooth synchronized scrolling.
- No excessive bouncing/karaoke effects in baseline design.
- Unavailable lyrics state is subtle and non-blocking.

8.3 Ambient mode — later

- Artwork-led cinematic background.
- Slow motion only.
- Minimal controls.
- No cloud asset dependency.

9. State, Failure & Recovery Model



Failure	Required behavior
Missing metadata	Continue playback; render available fields.
Artwork failure	Continue playback; use fallback visual.
Lyrics failure	Continue playback; Now Playing remains usable.
LRCLIB unavailable	Use local/cache or show no lyrics.
Low-confidence lyrics match	Reject rather than display likely-wrong lyrics.
Network interruption	Bounded retry/reconnect.

Sender disconnect	Clear session and return to Ready/Discovery.
Unsupported media	Clear compatibility message; no crash.
Malformed protocol input	Validate, isolate failure, log diagnostics.

10. Security, Privacy, Licensing & Legal Rules

- No analytics/telemetry by default.
- No account required.
- No mandatory internet connection for core receiver operation.
- Expose only network services required by the receiver.
- Use optional pairing/PIN mechanisms where supported.
- Do not log sensitive media payloads unnecessarily.
- Never implement DRM/FairPlay circumvention.
- Audit all third-party dependencies and licenses before integration.
- Preserve required third-party source/license notices.
- State clearly that OpenReceiver is independent and not Apple-endorsed.

UxPlay and other open-source receiver implementations are major references. Agents must inspect their implementation and license before reuse. Do not copy code blindly.

11. Repository Structure

```
OpenReceiver/  
├── core/  
│   ├── airplay/  
│   ├── mdns/  
│   ├── rtsp/  
│   ├── rtp/  
│   ├── pairing/  
│   ├── media/  
│   ├── metadata/  
│   ├── state/  
│   ├── lyrics/  
│   └── diagnostics/  
├── android/  
│   ├── app/  
│   ├── jni/  
│   └── ui/  
├── windows/  
│   ├── app/  
│   ├── native/  
│   └── ui/  
├── tests/  
│   ├── core/  
│   ├── metadata/  
│   ├── lyrics/  
│   ├── state/  
│   └── integration/  
├── docs/  
│   ├── architecture/  
│   ├── protocol-notes/  
│   ├── decisions/  
│   ├── development/  
│   └── release/  
├── scripts/  
├── CMakeLists.txt  
├── README.md  
├── LICENSE  
└── THIRD_PARTY_NOTICES
```

12. Agentic Development Organization

Agent	Ownership	Output
Protocol Agent	mDNS, AirPlay, RTSP/RTP, pairing/session	Receiver/session implementation + tests
Media Agent	Audio/video transport and platform handoff	Reliable media pipeline
Music State Agent	TrackInfo, PlaybackState, ReceiverState	Normalized state API + tests
Lyrics Agent	LRC, provider abstraction, LRCLIB, synchronization	Lyrics engine + tests
Music UI Agent	Now Playing, Lyrics, Ambient	Premium UI
Android TV Agent	Kotlin/JNI/NDK/Compose TV	TV integration
Windows Agent	C#/WinUI/native/GStreamer/WASAPI	Windows integration
QA Agent	Regression, compatibility, failure scenarios	Test matrix
Release Agent	Packaging, notices, docs	Release artifacts

Every agent task must be bounded and contain: objective, context, files/components, verified assumptions, implementation, tests, acceptance criteria, non-goals, documentation changes and known limitations.

13. Final Development Roadmap

Phase	Goal	Exit criterion
0 — Foundation	Repo/build/licensing/docs	Clean build + test harness
1 — Protocol proof	Discovery/session/basic audio	Real sender connects and audio works
2 — Music state	Metadata/artwork/playback normalization	Correct TrackInfo + PlaybackState
3 — Premium UI	Windows-first cinematic Now Playing	Beautiful real-data UI
4 — Lyrics	LRC + LRCLIB + synchronization	Lyrics follow play/pause/seek
5 — Windows	Tray/lifecycle/package	Installable Windows receiver
6 — Android TV	JNI/media/Compose TV	Installable TV receiver
7 — Mirroring/video	Broader media support	Compatible scenarios without regression
8 — Reliability	Caching/reconnect/performance	Soak/regression pass
9 — v1 release	Docs/notices/packages	Public reproducible release

14. Initial Agent-Ready Task Backlog

- Create repository skeleton, CMake/build system, licensing policy, ADR template and CI/test harness.
- Audit UxPlay and relevant open-source AirPlay receiver implementations. Document protocol behavior, reusable components, limitations and licenses.
- Build Windows AirPlay discovery/receiver proof-of-concept.
- Implement/verify AirPlay session lifecycle and supported audio path.
- Create TrackInfo, PlaybackState and ReceiverState models with unit tests.
- Normalize AirPlay metadata and artwork; add artwork cache.
- Expose playback timeline/state to platform shells.
- Build Windows Now Playing UI against mock state, then connect real state.
- Implement artwork-derived background generation and caching.
- Implement robust LRC parser and timestamp tests.
- Implement LocalLrcProvider.
- Implement LrclibProvider with safe matching, cache, timeout and non-fatal failure handling.
- Implement LyricsSynchronizer driven by playback position.
- Implement Lyrics UI mode and unavailable/error states.
- Add Windows tray/lifecycle/installer behavior.
- Build Android TV JNI bridge and native-core lifecycle integration.
- Implement Android TV media output and cinematic Now Playing UI.
- Run regression tests for metadata, pause, seek, disconnect, reconnect, artwork failure and lyrics failure.
- Only after music is stable, begin mirroring/photo/video work.
- Prepare release docs, troubleshooting, notices, limitations and reproducible packaging instructions.

15. Test Strategy

Layer	Examples
Unit	LRC timestamps, metadata normalization, state transitions, matching normalization
Protocol	Discovery, session setup/teardown, malformed messages
Media	Start/stop, pause, seek, reconnect, unsupported path
Lyrics	Exact match, mismatch, no result, malformed LRC, pause/seek sync
UI	Now Playing, missing artwork, lyrics unavailable, auto-hide
Integration	Real iPhone/Apples Music sender → OpenReceiver → UI
Soak	Long playback, repeated pause/seek, reconnect, track changes
Platform	Windows 10/11 and target Android TV/Google TV

Minimum music milestone: real sender selects OpenReceiver → audio plays → title/artist/artwork appear → pause/resume/seek/track change work → disconnect/reconnect works → lyrics failure does not affect audio.

16. Performance & UX Requirements

- Lyrics lookup is asynchronous and never blocks media playback.
- Artwork decoding/background generation is cached and off the critical path.
- Playback-position events are bounded and efficient.
- Caches have limits and cannot grow indefinitely.
- Retry loops are bounded and observable.
- UI remains responsive when providers are slow/offline.
- TV UI is readable from distance and remote-control friendly.

17. User Configuration

- Receiver display name.
- Lyrics enable/disable.
- Lyrics provider preference/order.
- Local LRC directory/import where supported.
- Cache size/retention.
- Background visual intensity.
- Control auto-hide behavior.
- Diagnostic/log level.
- Optional supported pairing/security controls.

18. Final Architectural Decisions Record

Decision	Final choice
Receiver targets	Android TV / Google TV + Windows
Shared core	C/C++
Android	Kotlin + JNI + NDK + Compose TV
Windows	C# + WinUI 3 + native C++
Media	Platform APIs + GStreamer where appropriate
Cloud/database	None required
Lyrics architecture	Provider abstraction
Automatic lyrics	LRCLIB first
Offline lyrics	Local .LRC
AirPlay lyric text	Do not assume it exists in receiver metadata
Default UI	Cinematic Now Playing
Mirroring/video	Later
DRM/FairPlay	No circumvention
Telemetry	Off by default

19. Non-Negotiable Rules for Coding Agents

- Never invent undocumented AirPlay behavior. Verify against upstream code, protocol documentation, packet captures or reproducible tests.
- Do not redesign the architecture while implementing a bounded task.
- Preserve shared-core/platform-shell separation.
- Do not introduce paid APIs or mandatory hosted services.
- LRCLIB is a provider implementation, not a permanent architectural dependency.
- Never block audio playback on metadata, artwork or lyrics.
- Never implement DRM/FairPlay circumvention.
- Do not add cloud infrastructure merely because it is convenient.
- UI consumes normalized state, not raw protocol fields.
- Every non-trivial feature needs tests and acceptance criteria.
- Document uncertainty instead of guessing.
- Audit licenses before integrating upstream code.
- Keep v1 narrow: stable music receiving before broad media features.

20. Definition of Done — v1

- OpenReceiver is discoverable on the local network as an AirPlay receiver.
- A compatible sender establishes a session and sends supported audio.
- Track metadata and artwork populate correctly for supported sessions.
- Play/pause/seek/track changes/disconnect/reconnect behave correctly.
- Windows provides the intended cinematic Now Playing experience.
- Android TV / Google TV provides the intended TV-optimized experience.
- Lyrics mode displays synchronized LRC lyrics when a suitable match exists.
- LRCLIB lookup is asynchronous, cached, failure-tolerant and non-essential to audio.
- Local .LRC fallback works.
- Missing metadata/artwork, lyric failures and disconnects are graceful.
- No mandatory cloud/backend/database exists.
- Third-party notices/licenses are complete.
- README, setup, troubleshooting, limitations and independent-project language are complete.

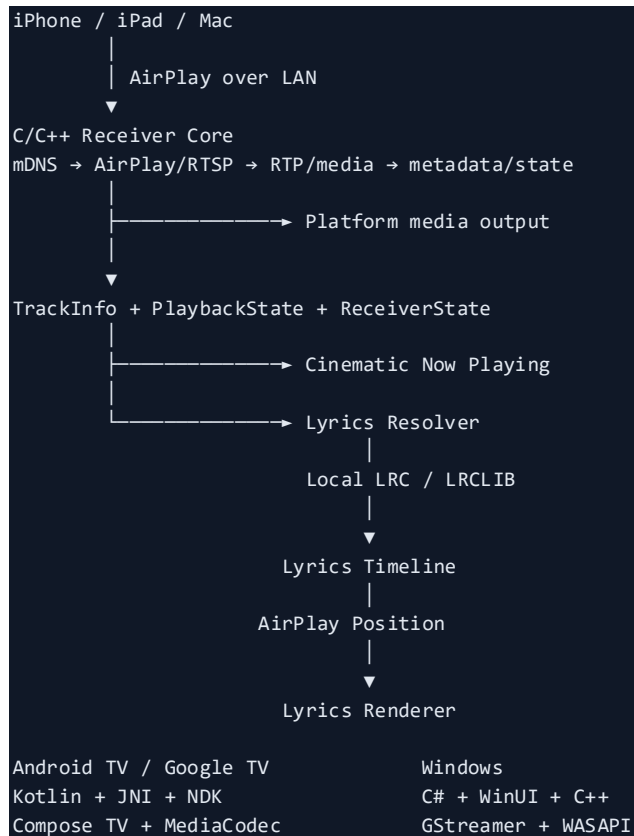
21. Future Expansion — Only After v1

- Screen mirroring improvements.
- Photo receiving/slideshow.
- Video receiving and compatible HLS scenarios.
- Additional lyrics providers and word-level synchronization.
- Ambient visual mode.
- Remote/device integrations.
- Additional receiver platforms if community demand exists.
- Optional provider/plugin system while preserving local-first operation.

22. Final End-to-End Blueprint

OpenReceiver = local-first AirPlay receiver + premium music experience.
--

OPENRECEIVER • ULTIMATE FINAL AGENTIC DEVELOPMENT SPECIFICATION



Engineering principle: AirPlay handles transport; the native core handles protocol/media state; platform shells handle device integration; the experience layer turns normalized state into the premium receiver UI.

Product principle: users should not need to know any of that. They select OpenReceiver, play music, and get a polished, reliable experience.

Appendix A — Agent Task Template

```

TASK ID:
TITLE:

OBJECTIVE:
One bounded outcome.

CONTEXT:
Relevant architecture and decisions.

FILES / COMPONENTS:
Exact areas expected to change.

ASSUMPTIONS:
Verified facts only; mark unknowns.

IMPLEMENTATION:
Smallest correct change.

TESTS:
Unit / integration / manual verification.

ACCEPTANCE CRITERIA:
Observable pass/fail conditions.

NON-GOALS:
What must not change.

DOCS:
Required documentation/decision updates.

RESULT:
Summary + tests + known limitations.

```

Appendix B — Release Checklist

- Clean checkout builds successfully.
- Automated tests pass.
- Windows install/package path verified.
- Android TV APK install path verified.
- Real sender interoperability verified.
- Lyrics provider failure proven non-fatal.
- No mandatory cloud/API key exists.
- Third-party notices reviewed.
- README/setup/troubleshooting complete.
- Known protocol/media limitations documented.
- No Apple endorsement or DRM circumvention claims.

Appendix C — Agent Definition of Behavior

Agents are implementation workers, not product decision-makers. When an agent encounters a protocol ambiguity, licensing ambiguity, unsupported media case, or architecture conflict, it must stop short of inventing behavior: document the ambiguity, identify evidence, propose the smallest change, and preserve the existing architecture unless an explicit decision is made.